How XPPAUT-Web works

Running the real XPPAUT, a 1990s X11 program written in C, inside a web browser

> **In one sentence:** the original XPPAUT source code, AUTO included, is compiled to WebAssembly. The X11 graphics system it was written for is replaced by a small library that draws into memory, and the web page shows each of XPP's windows as a canvas. Nothing is rewritten, so the numbers and the windows are the same as on the desktop.

Part	What it is	Size
XPPAUT 8.0 (with AUTO, CVODE)	Bard Ermentrout's C source, compiled almost unchanged	100 C files, about 94,000 lines
Xlib replacement (`web/xlib/`)	Windows, drawing, fonts, colours, events: the 69 X11 functions XPP uses	about 2,700 lines, plus generated font and colour tables
Web page (`src/`)	Toolbar, window panels, files, phones and tablets, offline app	about 2,200 lines of TypeScript
What a visitor downloads	The page, plus one WebAssembly file	about 0.5 MB compressed

1. The layers

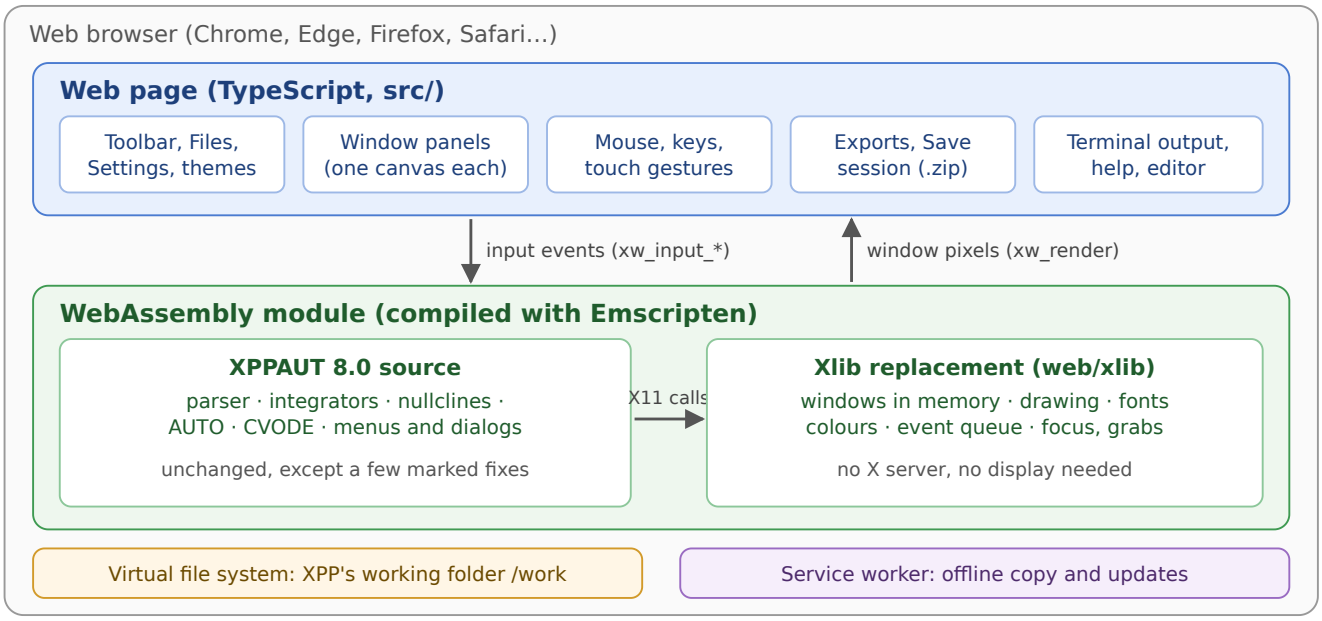


Figure 1. XPP's own code runs inside the WebAssembly module and believes it talks to an X11 display. The Xlib replacement keeps its windows in memory; the page shows them, and sends mouse, keyboard and touch input back.

Why compile the original instead of rewriting it? XPPAUT has evolved over 30 years. A rewrite would take years and would never match it exactly. Compiling the real code gives every feature at once: all integrators, AUTO, the file formats, the menus. It also means results can be checked against the desktop version number for number.

2. Replacing X11: from X windows to canvases

On Linux, XPP draws through **Xlib**, the client library of the X Window System: it creates windows, draws lines, text and polygons, and waits for events such as clicks and key presses. A browser has no X server, so `web/xlib/` implements those same Xlib functions itself. XPP is compiled against it without knowing the difference.

- **Windows** form a tree, as in X. Each window has a pixel buffer, a background, a border and a position. When something is drawn, it goes into the buffer of that window.
- **Drawing** follows the exact X11 rules, so lines, wide lines, arcs and polygons light up the same pixels as on a real X server. This is why the windows match the desktop pixel for pixel.
- **Fonts** are the classic X bitmap fonts (misc-fixed), converted into tables at build time. **Colours** use the X colour names and a true-colour display.
- **Events**: clicks, keys, Expose (redraw) and Enter/Leave go through a queue, with pointer grabs and click-to-focus as XPP expects.
- Each **top-level window** (main window, AUTO, Parameters, Data browser…) is composited from its child windows into one image, which the page paints into its own canvas panel.

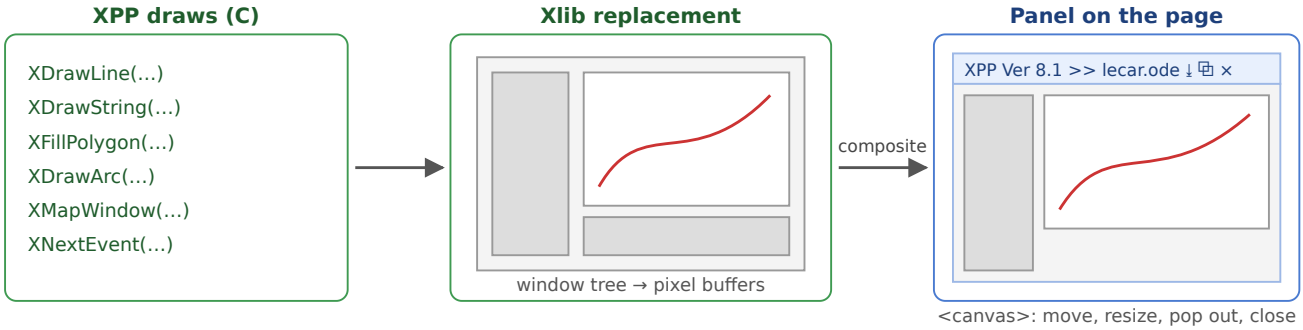


Figure 2. XPP's drawing calls land in in-memory windows; each top-level window becomes one canvas panel that the page can move, resize, save as PNG or pop out into a separate browser window.

3. Running a blocking program in a browser

Desktop XPP is written as an endless loop: it waits for the next event, handles it, and waits again. Long computations (integrating, AUTO) run without returning. A web page cannot do that: if code never gives control back, the page freezes and nothing is painted. XPP's loop therefore has to pause regularly and give the browser a turn.

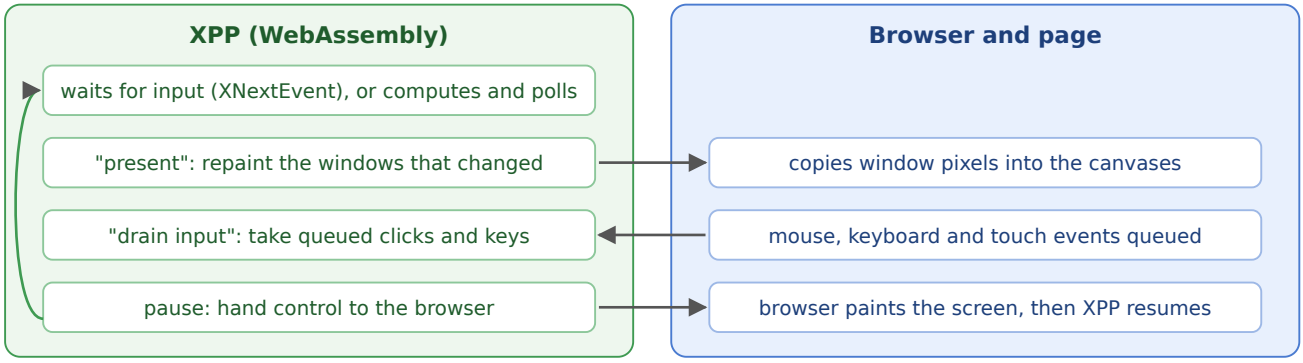


Figure 3. Whenever XPP waits, and about 30 times a second while it computes, the Xlib layer repaints the changed windows, collects input, and pauses so the browser can paint and stay responsive.

Pausing in the middle of C code needs help from the compiler. The site ships the same program in two builds and picks one automatically:

Build	How it pauses	Used by	Size (compressed)
JSPI (fast)	The browser suspends WebAssembly natively (JavaScript Promise Integration)	Recent Chrome and Edge	about 0.44 MB
Asyncify (fallback)	The compiler rewrites the code so it can save and restore its own stack	Firefox, Safari, older browsers	about 0.57 MB

4. Files and exports

XPP reads and writes ordinary files. In the browser they live in a virtual file system in memory, with XPP's working folder at `/work`. The model you open (and its `.ani`, `.tab`, `.dat` files) is placed there before XPP starts.

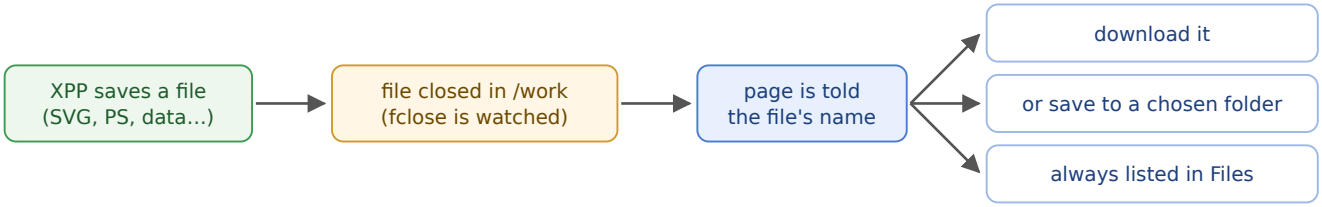


Figure 4. Every file XPP finishes writing is noticed and delivered as chosen under Settings → Exports. AUTO's internal work files are skipped.

Save session collects the working folder, a screenshot of every open window (folder `SS`) and XPP's terminal output into one `.zip`, built in the browser. Nothing is uploaded: all computation and all files stay on the user's device.

5. The page around XPP

- **Window panels**: the page acts as XPP's window manager: it places, raises, moves and resizes windows, and tells XPP when a window was resized or closed. ⧉ moves a panel into a separate browser window.
- **Start screen**: 212 example models in four groups, plus your own files; the WebAssembly file starts downloading as soon as the page opens, with a progress bar.
- **Terminal output** (hidden, side panel or separate window), the **model editor**, XPP's **help** pages, and ten **colour themes** for the page.

6. Phones and tablets

Touch devices are recognised by what they can do (no hover, a coarse pointer), not by browser name. All of XPP's windows sit on a "world" layer that can be zoomed and moved, and touches are translated into the mouse and keyboard events XPP understands:

On the screen	XPP receives
Tap · drag · long-press	Left click · left-button drag · right click
Pinch · two-finger drag · Fit	Nothing: zoom and pan the view
Two-finger tap · Android Back	Esc
Cmd panel · ⌨ keyboard and key strip	The keys of XPP's commands (I G, N N, …), Esc, Tab, Ctrl, arrows

On phones each XPP window fills the width of the screen, and tabs switch between windows.

7. Installable app and offline use

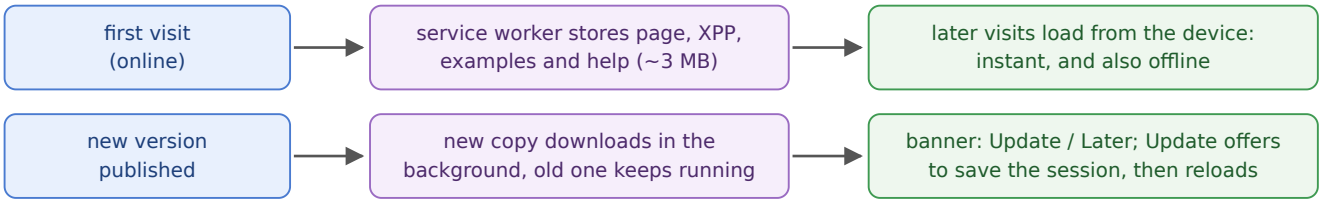


Figure 5. A web app manifest makes the site installable (its own icon and window). The service worker keeps a versioned offline copy; each published change gets a new version, which the app offers as an update.

8. Build and publishing

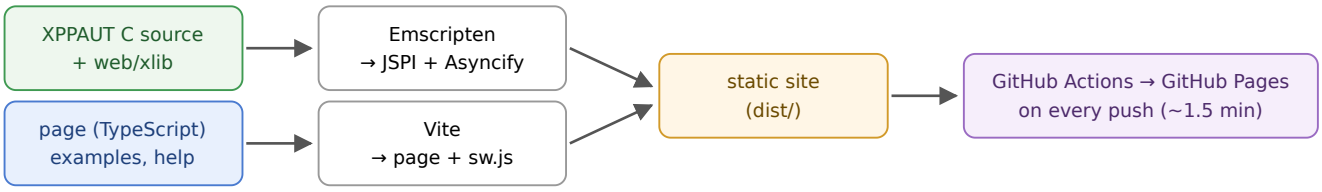


Figure 6. The whole site is static files: no server-side code, so many people can use it at once at no cost.

9. How it is tested

- **Numbers**: 144 example models are run in batch mode by the desktop XPPAUT and by the web build; the results are compared and are identical, for both builds.
- **Pixels**: 26 scripted sessions click and type through XPP (menus, AUTO, nullclines, file dialogs, 3D views, animation…) on the real desktop XPPAUT under a virtual X display and on the web version; 38 screenshots are compared pixel by pixel and are identical.
- **The page**: headless Chrome drives the site, including phone and tablet emulation, touch gestures, pop-out windows, downloads and offline use.

10. Changes to XPPAUT itself

Only small, marked fixes were needed, all listed in `xppaut/UPSTREAM.md`: a few bugs that desktop compilers tolerate but WebAssembly does not (mismatched function declarations, a one-byte buffer overflow in `export`, a typo that passed a library function as a flag), opening the HTML help in a browser tab, and AUTO fixes from Bard Ermentrout's 2025 version.

XPPAUT-Web by Jeevan Kumar Gochhayat · zetaone.github.io/XPPAUT-Web · source: github.com/ZetaOne/XPPAUT-Web (GPL v2). XPPAUT © Bard Ermentrout; AUTO by E. J. Doedel; CVODE by S. D. Cohen and A. C. Hindmarsh (LLNL).